Reusable Bare-Metal USART Functions

STM32CubeIDE reference guide | Register-level code structure | Interrupt-driven receive

Goal: Build a reusable USART driver so future bare-metal projects can reuse the transmit, receive-buffer and interrupt code. Usually, only the application-specific command or message handler changes.

This guide documents the nine functions from your selected list, how they fit together, and which parts must be adapted for a different STM32 model or USART peripheral.

USART hardware Receives a byte	**USART_IRQHandler()** Reads status/data	**RingBuffer_Push()** Stores byte in RAM	**RingBuffer_Pop()** Main code retrieves it	**USART_ProcessRx()** Builds a message

Important: There is no single implementation that is identical for every STM32 project. Register names, interrupt vector names, GPIO pins and clock calculations depend on the MCU family and selected USART instance. The patterns below are reusable, but initialization and the ISR need target-specific adaptation.

1. The nine reusable functions at a glance

No.	Function	Purpose	Reuse note
1	**USART_Init()**	Configures peripheral clock, TX/RX pins, baud rate, frame format and interrupt settings.	Reuse the design; adapt registers, pins and clock.
2	**USART_IRQHandler()**	Handles the USART interrupt, reads received data and handles relevant error flags.	Must match the selected MCU and peripheral vector.
3	**RingBuffer_Push()**	Places a newly received byte into the ring buffer. Usually called by the ISR.	Reusable when buffer rules and data ownership match.
4	**RingBuffer_Pop()**	Returns the oldest unread byte to main code.	Reusable with the same ring-buffer structure.
5	**USART_SendByte()**	Waits for the transmit data register to be ready and writes one byte.	Adapt register/status flag names to MCU family.
6	**USART_Write()**	Transmits a byte array or buffer of a given length.	Reusable; may later be changed to interrupt/DMA TX.
7	**USART_WriteString()**	Transmits a null-terminated text string.	Useful for text output; not needed for binary-only protocols.
8	**USART_ProcessRx()**	Reads buffered bytes outside the ISR and assembles messages or frames.	Reusable for the same protocol/framing; adapt parser when protocol changes.
9	**USART_GetRxErrors()**	Reports error counters such as overrun, framing, noise, parity and software buffer drops.	A recommended API; add it if the driver needs diagnostics.

2. Function-by-function reference

1) USART_Init()

Purpose: Initializes the UART peripheral before the application starts.

Typical implementation: Enable the peripheral clock; configure TX/RX GPIO alternate functions; calculate BRR from the real peripheral clock; set data bits, parity and stop bits; configure RX interrupt and NVIC; enable USART, transmitter and receiver.

Reuse note: Must be adapted to the chosen STM32 family, USART number, pins, clock tree and desired baud rate.

2) USART_IRQHandler()

Purpose: Runs automatically when an enabled USART interrupt condition occurs.

Typical implementation: Read status flags, read the data register as required by the MCU reference manual, record relevant errors, and pass valid received data to RingBuffer_Push(). Keep it short.

Reuse note: The exact ISR symbol and error-clearing sequence differ across STM32 families. Define it only once.

3) RingBuffer_Push()

Purpose: Stores a received byte at the producer index (head).

Typical implementation: Compute the next head index; detect a full buffer; write the byte; publish the new head index. Record dropped bytes when full.

Reuse note: Normally called by the ISR in a single-producer/single-consumer design. Choose a power-of-two size if using bit masking.

4) RingBuffer_Pop()

Purpose: Reads the oldest available byte from the consumer index (tail).

Typical implementation: Return false when empty; copy the oldest byte to the caller; advance tail and wrap at the end.

Reuse note: Normally called by main code. Use clear ownership rules for indexes shared between ISR and main.

5) USART_SendByte()

Purpose: Transmits a single byte using polling in this basic driver.

Typical implementation: Wait until TXE/TXFNF indicates the transmit data register/FIFO can accept data; write the byte to the correct transmit register.

Reuse note: Status bit names differ by STM32 family. Avoid waiting forever in time-critical code without a timeout strategy.

6) USART_Write()

Purpose: Sends a buffer with a known length.

Typical implementation: Loop through the buffer and send each byte, or use a TX interrupt/DMA implementation for higher throughput.

Reuse note: Length-based transmission supports text and binary data. Define who owns a buffer if transmission is asynchronous.

7) USART_WriteString()

Purpose: Convenience function for ordinary C strings.

Typical implementation: Call USART_Write() with the string bytes and a length from strlen().

Reuse note: Only for null-terminated strings. Do not use strlen() on arbitrary binary buffers.

8) USART_ProcessRx()

Purpose: Moves bytes from the ring buffer to a protocol parser in normal execution.

Typical implementation: Pop available bytes; recognize a delimiter or frame length; validate size; terminate or dispatch a complete message; handle malformed or overlong input.

Reuse note: The line-based example uses newline framing. Binary protocols usually need a length field, delimiter/escaping, or another framing method.

9) USART_GetRxErrors()

Purpose: Provides application code a way to inspect receive quality and buffer loss.

Typical implementation: Return a snapshot of counters such as overrun, framing, noise, parity and software ring-buffer drops. If counters are shared with an ISR, take a safe snapshot.

Reuse note: This was in the reusable-function list as a recommended function; it must be implemented in your driver if you need it.

3. Recommended reusable driver interface

Use a header such as **usart_driver.h** to expose only the functions the application needs. Keep ring-buffer details and private variables inside **usart_driver.c**.

```
/* usart_driver.h - example public interface */
#ifndef USART_DRIVER_H
#define USART_DRIVER_H

#include <stdint.h>
#include <stddef.h>
#include <stdbool.h>

/* Hardware-specific: configure selected USART and RX interrupt. */
bool USART_Init(uint32_t baudRate);

/* Blocking/polling transmit API. */
void USART_SendByte(uint8_t byte);
void USART_Write(const uint8_t *data, size_t length);
void USART_WriteString(const char *text);

/* Called from main-loop context to process received bytes. */
void USART_ProcessRx(void);

/* Optional diagnostics snapshot; define an error structure. */
void USART_GetRxErrors(/* USART_ErrorStats *out */);

#endif
```

Example application usage

```
int main(void)
{
    USART_Init(115200U);
    USART_WriteString("USART ready\r\n");

    while (1)
    {
        USART_ProcessRx();
        /* Application tasks run here. */
    }
}
```

In the interrupt handler, received bytes are pushed to the ring buffer. Do not parse long commands, print lengthy messages or run blocking application logic inside the ISR.

4. What changes between projects?

Usually reusable	Must be checked or adapted
Ring-buffer algorithm; length-based write API; string convenience API; parser structure when the framing is unchanged.	Clock setup; USART instance and IRQ name; GPIO pins; baud-rate calculation; status/error flag handling; interrupt priority; message parser and application handler.

5. Practical checklist before reuse

- Confirm the exact STM32 part number and use its CMSIS device header and reference manual.
- Confirm the actual peripheral clock feeding the selected USART; calculate the baud-rate register from that clock.
- Check the selected TX/RX pins and alternate-function configuration in the device datasheet.
- Verify the ISR name and error-flag clearing sequence for the chosen MCU family.
- Choose a ring-buffer size based on the highest incoming rate and the longest time the main loop may be busy.
- Test normal traffic, a full ring buffer, an overlong command, malformed input and UART error conditions.

Key takeaway: Reuse the driver architecture, not blindly the register values. Keep hardware-specific initialization and interrupt details separate from the application protocol. This is a sound modular design pattern; individual companies may use different naming, layering, DMA, RTOS or coding-standard requirements.